CHAPTER THREE

SYSTEM ANALYSIS AND MODELING

3.1 Overview

System analysis and modeling is a critical phase in software development that focuses on understanding system requirements and translating them into structured models. This phase ensures that the proposed system is well-defined, logically organized, and capable of meeting user and organizational needs before actual implementation begins.

In this chapter, the UniMap – Smart University Map and Navigation System is analyzed in detail to identify system actors, system functionalities, data flow, and interactions between different components. Various modeling techniques are used to represent the system from different perspectives, including functional, structural, and behavioral views.

The chapter presents system models such as use case diagrams, system architecture, data models, and process diagrams to clearly illustrate how users interact with the system and how data is processed internally. These models serve as a blueprint for system development and provide a clear understanding of the system's operation for developers, supervisors, and stakeholders.

By applying standard system analysis and modeling tools, this chapter ensures that the UniMap system is scalable, maintainable, and aligned with the functional and non-functional requirements defined in the previous chapter.

3.2 Scenario-Based Modeling

Scenario-based modeling describes how users interact with the system through real-world usage scenarios. It focuses on identifying system actors and defining the actions they perform while using the UniMap system. This approach helps in understanding system behavior from the user's perspective and ensures that all functional requirements are properly captured.

Scenario-based modeling is mainly represented using use cases, which describe the interaction between users (actors) and the system to achieve specific goals. These use cases form the foundation for system design, implementation, and testing.

3.2.1 Use Case Identification

Use case identification involves recognizing the different types of users who interact with the system and defining the functions they can perform. For the UniMap – Smart University Map and Navigation System, the following actors and use cases have been identified.

Actors

- **Guest User:** Visitors or new students who use the system to find locations within the university campus.
- **Registered User (Student/Staff):** University members who use the system for navigation and accessing campus information.
- **Administrator:** Authorized personnel responsible for managing and maintaining the system data.

Identified Use Cases

- View campus map
- Search for buildings, offices, and departments
- View building or facility information
- Get walking route between two locations
- View campus facilities and services
- View event or announcement locations (optional)
- Log in to the system (Administrator)
- Add campus locations (Administrator)
- Update campus map data (Administrator)
- Delete outdated or incorrect map information (Administrator)

3.2.2 Actor Identification

Actors represent external entities that interact with the UniMap system. Each actor plays a specific role and has defined responsibilities within the system.

Identified Actors

1. **Guest User**

    - Includes visitors and new students.
    - Can access the system without registration.
    - Uses the system to view the campus map, search locations, and get navigation routes.

2. **Student/Staff User**

    - Includes registered students and university staff members.
    - Uses the system for campus navigation and accessing location information.

- Shares similar functionality with guest users but may access additional information services.

3. **Administrator**

    - Authorized university personnel.
    - Responsible for managing campus data.
    - Can add, update, and delete buildings, routes, facilities, and events.
    - Ensures accuracy and reliability of system data.

3.2.3 Use Case Description

This section provides textual descriptions of key use cases to explain how actors interact with the UniMap system.

Use Case 1: View Campus Map

- **Actor:** Guest User, Student/Staff
- **Description:** The user accesses the system and views the interactive campus map displaying buildings and facilities.
- **Precondition:** The system is available.
- **Postcondition:** The campus map is displayed on the user interface.

Use Case 2: Search Location

- **Actor:** Guest User, Student/Staff
- **Description:** The user searches for a specific building, office, or department using the search feature.
- **Precondition:** The system is available.
- **Postcondition:** The selected location is highlighted on the map.

Use Case 3: Get Route

- **Actor:** Guest User, Student/Staff
- **Description:** The user selects a starting point and destination to receive walking directions.
- **Precondition:** Valid locations are selected.
- **Postcondition:** A route is displayed on the map.

Use Case 4: View Facility Information

- **Actor:** Guest User, Student/Staff
- **Description:** The user views detailed information about campus facilities.

- **Precondition:** A facility exists in the system.
- **Postcondition:** Facility details are displayed.

Use Case 5: Manage Campus Data

- **Actor:** Administrator
- **Description:** The administrator logs into the system and manages campus map data.
- **Precondition:** Administrator authentication is successful.
- **Postcondition:** Campus data is updated in the system.

3.2.4 Activity Diagram

The activity diagram illustrates the flow of actions performed by users when interacting with the UniMap system. It represents the sequence of activities from the start of system usage to the completion of a task.

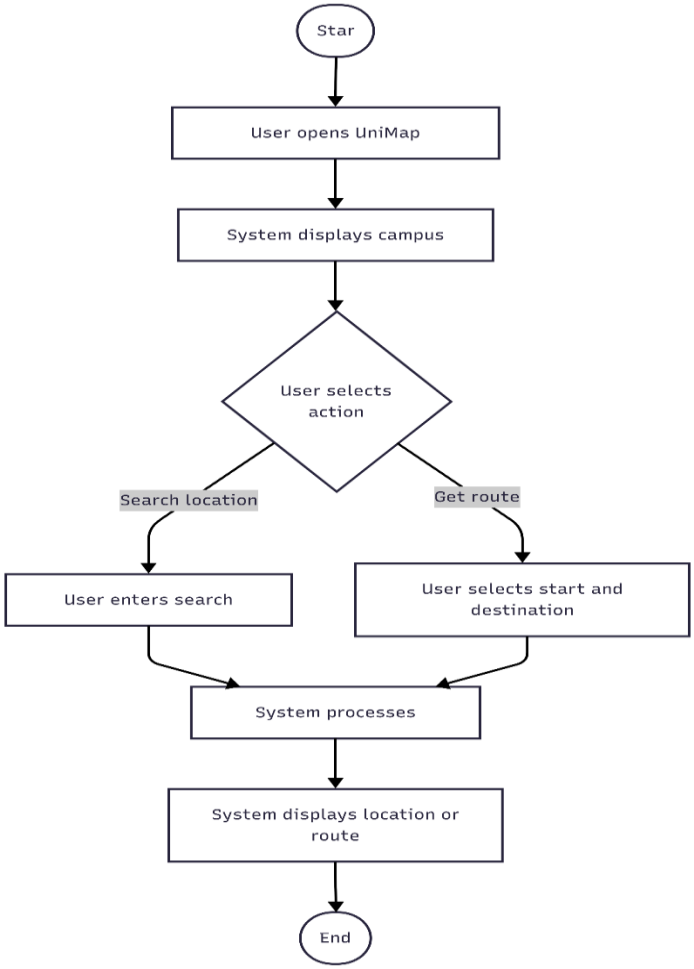

Figure 3.1: Activity Diagram of UniMap System

Description of Activity Flow

1. User opens the UniMap application.
2. System displays the campus map.
3. User selects an action (search location or get route).
4. User enters search details or selects locations.
5. System processes the request.
6. System displays the requested location or route.
7. User ends the session.

This activity flow represents the typical interaction between users and the UniMap system and will be illustrated using an activity diagram during system design.

3.3 Behavioral / Dynamic Modeling

Behavioral or dynamic modeling focuses on how the system behaves over time and how different components interact to accomplish a specific task. Unlike structural models, behavioral models emphasize the sequence of actions, message exchanges, and interactions between system actors and internal components.

In the UniMap system, behavioral modeling is used to illustrate how users interact with the application and how the system processes requests such as searching for locations and generating navigation routes. This modeling approach helps in understanding system logic, communication flow, and interaction order.

3.3.1 Sequence Diagram

A sequence diagram represents the interaction between actors and system components in a chronological order. It shows how messages are passed between the user interface, backend services, and the database to fulfill a specific use case.

For the UniMap – Smart University Map and Navigation System, the sequence diagram focuses on a typical navigation scenario, such as searching for a location and requesting a route within the campus.

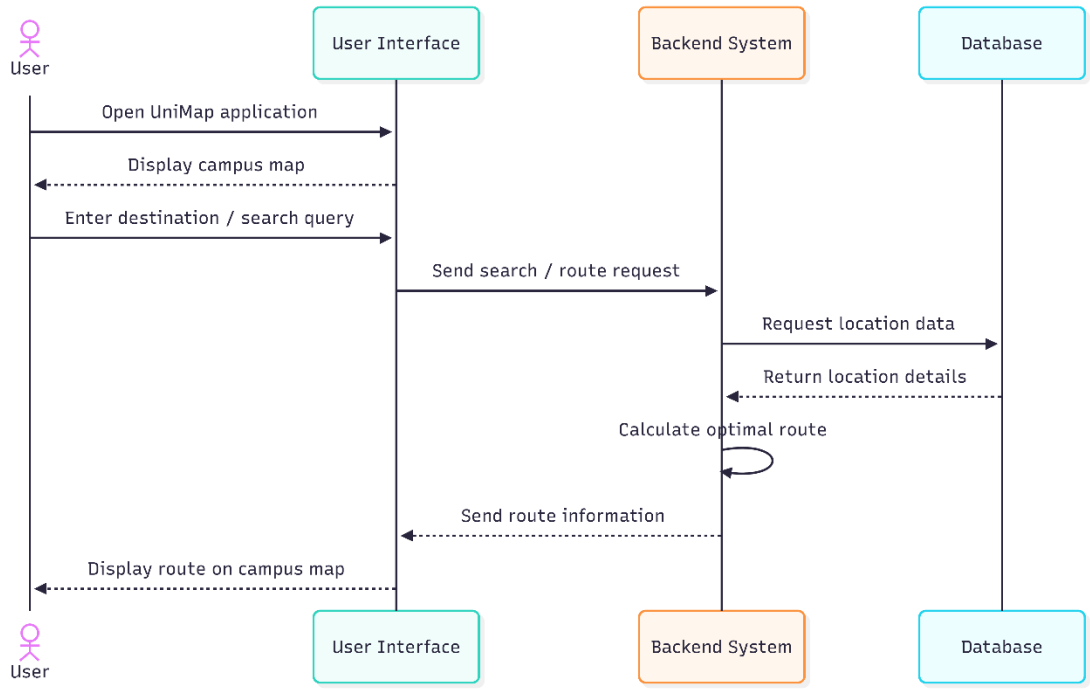

Figure 3.2: Sequence Diagram for Location Search and Routing in UniMap

Description of the Sequence Flow

1. The user opens the UniMap application.
2. The user selects a destination or enters a search query.
3. The user interface sends the request to the backend system.
4. The backend processes the request and retrieves relevant location data from the database.
5. The backend calculates the appropriate route.
6. The system sends the route information back to the user interface.
7. The user interface displays the route on the campus map.

This interaction sequence will be illustrated using a sequence diagram to clearly demonstrate the dynamic behavior of the UniMap system.

3.3.2 State Diagram

A state diagram represents the various states that the UniMap system can occupy and the transitions between those states based on user actions or system events. This type of diagram provides a clear view of how the system reacts to events, such as searching for a location, generating a route, or updating campus data.

In the UniMap system, state diagrams are useful for modeling both **user interactions** and **administrative actions**, highlighting how the system moves from one state to another to fulfill specific functionalities.

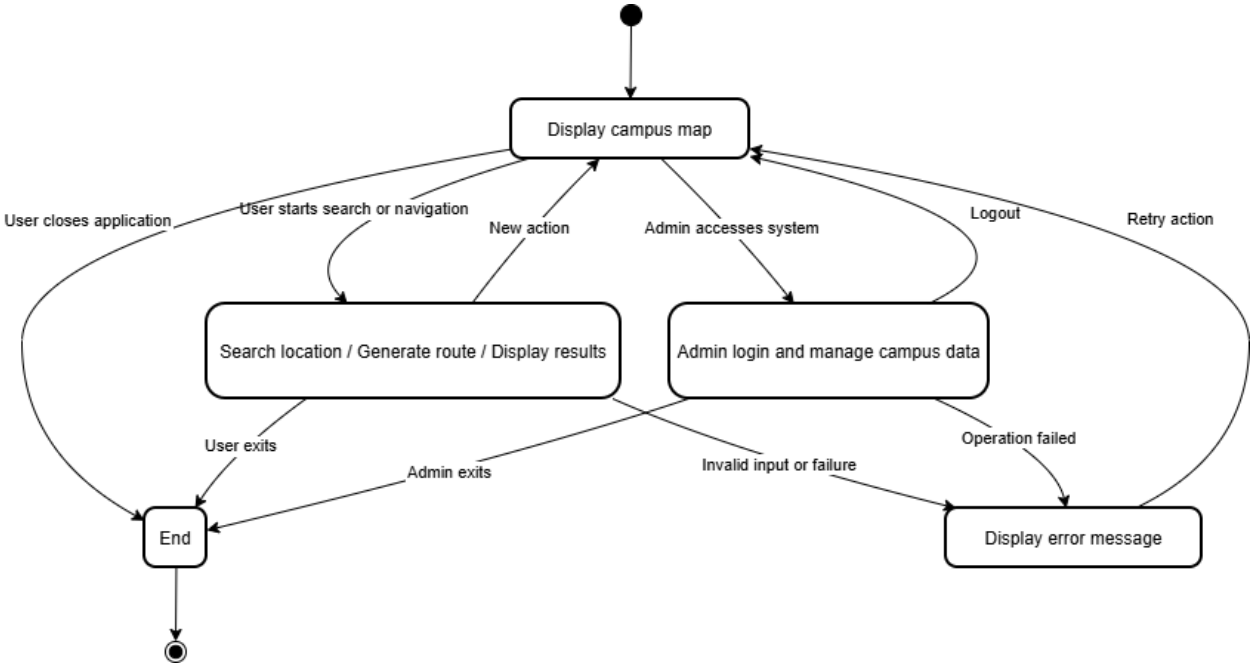

Figure 3.3: Simplified State Diagram of the UniMap System

Key States of the UniMap System

Idle / Home State

- The system is initialized and displays the main campus map.

User Navigation State (merged)

Represents the combined flow of:

- Searching for a location
- Generating a route
- Displaying results on the map

Admin Management State (merged)

- Administrative users authenticate and manage campus data (add, update, delete locations, routes, facilities).

Error / Exception State

- The system handles invalid inputs or failed operations and provides feedback to the user.

End / Exit State

- The user closes the application or logs out.

Transitions Between States

- From **Idle → User Navigation**: When a user initiates a search or route request.
- From **Idle → Admin Management**: When an administrator accesses the system.
- From **User Navigation → Idle**: After completing an action, the system returns to idle.
- From User **Navigation → Error**: If an invalid input or failure occurs.
- From **User Navigation → End**: When the user closes the application during navigation.
- From **Admin Management → Idle**: After admin logs out.
- From **Admin Management → Error**: If an operation fails.
- From **Admin Management → End**: When the admin exits the system.
- From Error → **Idle**: After resolving the issue or retrying.
- Idle **/ Any active state → End**: When the session ends.

3.4 Class-Based Modeling

Class-based modeling describes the system's static structure by identifying the main classes, their attributes, methods, and relationships. This type of modeling is crucial for object-oriented design and implementation, as it guides database design, API structure, and frontend-backend integration.

In the UniMap system, class-based modeling is used to define the entities involved in campus navigation and management, their properties, and how they interact with each other.

3.4.1 Identifying Classes

Based on the functional and non-functional requirements, the following key classes have been identified for the UniMap system:

1. User

- **Attributes:**
    - `user_id` (unique identifier)
    - `name`
    - `email`

    - `role` (Guest, Student, Staff, Admin)
    - `password` (for Admin users)

- **Methods:**

    - `login()`
    - `logout()`
    - `viewProfile()`
    - `updateProfile()` (for registered users)

2. Location

- **Attributes:**

    - `location_id` (unique identifier)
    - `name` (building, office, facility)
    - `type` (academic, administrative, facility)
    - `latitude`
    - `longitude`
    - `description`
    - `image` (optional)

- **Methods:**

    - `getDetails()`
    - `updateDetails()` (Admin only)
    - `deleteLocation()` (Admin only)

3. Route

- **Attributes:**

    - `route_id` (unique identifier)
    - `start_location_id`
    - `end_location_id`
    - `distance`
    - `path_coordinates` (array of GPS coordinates)

- **Methods:**

    - `calculateRoute()`
    - `displayRoute()`

4. Facility

- **Attributes:**
  - `facility_id`
  - `name`
  - `location_id` (associated with a building)
  - `description`
  - `image` (optional)
- **Methods:**
  - `getFacilityInfo()`
  - `updateFacility()` (Admin only)

5. Event **(Optional Module)**

- **Attributes:**
  - `event_id`
  - `title`
  - `location_id`
  - `date_time`
  - `description`
- **Methods:**
  - `getEventInfo()`
  - `addEvent()` (Admin only)
  - `updateEvent()` (Admin only)
  - `deleteEvent()` (Admin only)

6. Map

- **Attributes:**
  - `map_id`
  - `locations` (list of all campus locations)
  - `routes` (list of all routes)
- **Methods:**
  - `displayMap()`
  - `updateMapData()` (Admin only)

Relationships Between Classes

- **User ↔ Map:** Users interact with the map to view locations and routes.
- **Location ↔ Route:** Routes connect two or more locations.
- **Location ↔ Facility:** Facilities belong to specific locations.
- **Location ↔ Event:** Events are assigned to locations.
- **Admin ↔ All Classes:** Admin users manage Locations, Routes, Facilities, Events, and Map data.

3.4.2 Class Diagram

A class diagram is a **static structural model** that illustrates the system's classes, their attributes, methods, and relationships. It provides a clear blueprint for developers and helps stakeholders understand how different components of the system interact.

For the **UniMap – Smart University Map and Navigation System**, the class diagram represents the key entities involved in campus navigation and administration.

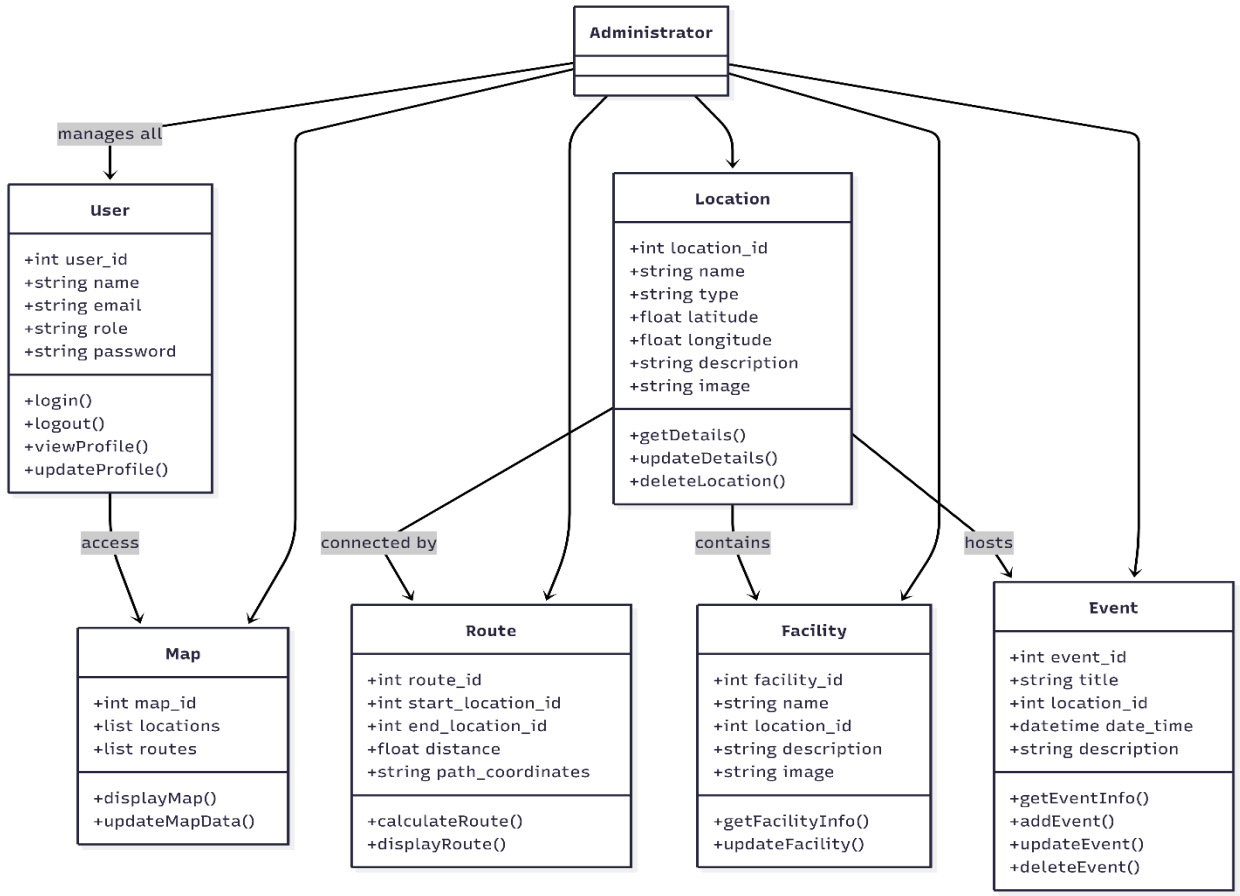

Figure 3.4: Class Diagram of the UniMap System

Table 3.1 System classes with there Attributes and Methods

Class	Attributes	Methods
User	user_id, name, email, role, password	login(), logout(), viewProfile(), updateProfile()
Location	location_id, name, type, latitude, longitude, description, image	getDetails(), updateDetails(), deleteLocation()
Route	route_id, start_location_id, end_location_id, distance, path_coordinates	calculateRoute(), displayRoute()
Facility	facility_id, name, location_id, description, image	getFacilityInfo(), updateFacility()
Event	event_id, title, location_id, date_time, description	getEventInfo(), addEvent(), updateEvent(), deleteEvent()
Map	map_id, locations, routes	displayMap(), updateMapData()

Relationships

- **User → Map:** Users access the map to view locations and routes.
- **Location → Route:** Routes connect locations.
- **Location → Facility:** Each facility belongs to a specific location.
- **Location → Event:** Events are associated with a location.
- **Administrator → All Classes:** Admin manages Locations, Routes, Facilities, Events, and Map data.